

A COURSE COMPANION



NumPy Book

Lessons & Explanations

A lesson-by-lesson walkthrough of **NumPy**, built from the YouTube course by **JoeTech**.

What's inside

01

Introduction to NumPy

3

JoeTech, NumPy Course, Video 1 — Introduction to NumPy

Introduction to NumPy

NumPy is a third-party Python module built to deal with **arrays** and **matrices**. The name is short for *Numerical Python*, it is fully **open source**, and it is designed to handle large, multidimensional arrays and matrices efficiently. On top of that storage, NumPy ships a large collection of **mathematical functions** made to operate on those elements directly.

Being *third-party* simply means it doesn't come bundled with Python — you install it yourself before using it. Its source code is public on GitHub at github.com/numpy/numpy.

بالعربي

يعني إيه NumPy؟ ببساطة هي **مكتبة خارجية** في بايثون، شغلتها الأساسية إنها تتعامل مع **المصفوفات** (Matrices و Arrays). الاسم نفسه اختصار لـ *Numerical Python*، يعني «بايثون الحسابية».

وهي **Open Source**، يعني الكود بتاعها مفتوح للكل على GitHub، أي حد يقدر يشوفه ويشارك فيه.

كلمة **Third-Party** معناها إنها مش جاية مع بايثون من الأول — لازم تنزلها بنفسك (هنشوف إزاي في آخر الدرس).

وأهم حاجة: هي مش بس بتخزن الأرقام، لأ ده كمان جاية معاها **دوال رياضية كتير جدًا** جاهزة تشتغل على الأرقام دي على طول.

Why do we use a NumPy array?

You could store numbers in a normal Python list — so why does NumPy exist at all? Because a NumPy array gives you five concrete wins over a list:

ADVANTAGE	WHAT IT MEANS
Consumes less memory	The same numbers take up noticeably less RAM
Very fast compared to a Python list	Operations run far quicker, especially on big data

ADVANTAGE

WHAT IT MEANS

Easy to use	Clean, short syntax for things that would need loops otherwise
Supports element-wise operation	One operation applies to every element at once
Elements are stored contiguous	All elements sit next to each other in memory

That last point — **contiguous storage** — is the reason the first four are true. In a Python list the values are scattered around memory and the list only keeps addresses pointing at them, so the computer has to jump around to read them. A NumPy array puts all its values side by side in one unbroken block, so reading them is fast and there's no per-value overhead to pay for.

بالعربي

طب ليه أصلاً نستخدم ال Array بتاعة NumPy وما نستخدمش ال List العادية؟ عشان بتديك ٥ مكاسب:

- **بتأخذ ذاكرة أقل** — نفس الأرقام بتحجز رامات أقل بكثير.
- **أسرع بكثير من ال List** — وكل ما البيانات تكبر، الفرق يبان أكثر.
- **سهولة في الاستخدام** — كود قصير وبسيط بدل لفة (Loop) طويلة.
- **بتدعم ال Element-Wise Operation** — يعني تعمل عملية واحدة وتتطبق على كل العناصر مرة واحدة من غير لوب خالص.
- **العناصر مخزنة ورا بعضها (Contiguous)** — وده السبب الحقيقي في كل اللي فوق.

خلينا نفهم نقطة ال **Contiguous دي كويس**، لأنها مفتاح الموضوع كله:

تخيل ال **List** زي كشكول فيه ورقة مكتوب فيها عناوين بيوت — الأرقام نفسها مش في الكشكول، هي متفرقة في أماكن مختلفة في الذاكرة، والكشكول بس شاليل العناوين. فعشان الكمبيوتر يقرأ رقم، لازم يروح للعنوان الأول، يرجع، يروح للتاني... لفة كتير ووقت ضايع.

أما ال **Array** بتاعة NumPy فهي زي شريط الشيكولاتة — كل القطع جنب بعض في بلوك واحد متواصل. الكمبيوتر بيقرا ورا بعضه على طول من غير لفة.

وعشان كده هي أسرع، وبتأخذ مساحة أقل (مفيش عناوين زيادة تتخزن).

Homogeneous vs. heterogeneous

Two terms worth knowing before going further, because they describe the core difference between a Python list and a NumPy array:

- **Homogeneous** — a collection that can contain only objects of *the same* type.
- **Heterogeneous** — a collection that can contain objects of *different* types.

A Python list is heterogeneous: you can freely mix an integer, a string, and a float in the same list. A NumPy array is not — **the items in the array have to be of the same type**.

This restriction is not a downside; it is exactly what buys you the advantages above. Because every element is guaranteed to be the same type, NumPy knows precisely how many bytes one element occupies, which means **you can be sure what storage size is needed for the array** before it is even created. A Python list can never promise that, because it has no idea what you might put in it next.

One last detail to keep in mind from the start: **NumPy arrays are indexed from 0** — the first element sits at position `0`, exactly like Python lists.

بالعربي

في كلمتين هتقابلهم كثير، خيلنا نفهمهم:

- **Homogeneous** — يعني «متجانس»: كل العناصر اللي جواه من نفس النوع.
- **Heterogeneous** — يعني «غير متجانس»: ممكن يشيل أنواع مختلفة مع بعض.

ال **List** في بايثون هي Heterogeneous — تقدر تحط فيها رقم وكلمة وكسر عشري كلهم مع بعض عادي جداً ومحدث هيزعل منك.

لكن ال **Array** بتاعة NumPy لآ — كل العناصر لازم تكون من نفس النوع.

وده مش عيب فيها، ده بالظبط سرّ قوتها! ليه؟

لأنه طالما كل عنصر من نفس النوع، يبقى NumPy عارفة بالظبط العنصر الواحد بياخد كام بايت. وطالما عارفة كده، يبقى تقدر تحسب **المساحة المطلوبة للمصفوفة كلها** قبل ما تعملها أصلاً.

أما ال **List** فمستحيل تعرف، لأنها مش عارفة انت هتخط فيها إيه بعد كده — ممكن رقم، ممكن كلمة، ممكن أي حاجة.

وآخر حاجة مهمة: الترقيم في NumPy بيبدأ من **صفر** مش من واحد — أول عنصر مكانه `0`، زي ال **List** بالظبط.

Installing and importing

NumPy is third-party, so install it once from the terminal:

```
pip install numpy
```

Then, in your Python file:

```
import numpy as np

print(np.__version__)
```

شرح الكود سطر سطر

- `import numpy as np` — بنسندعي المكتبة، وبنديها اسم مختصر `np`. ليه الاختصار ده بالذات؟ لأنه الاتفاق المتعارف عليه بين كل مبرمجي بايثون في الدنيا. أي كود NumPy هتشوفه في أي حته هيكون مكتوب بـ `np`، فاتعوّد عليه من دلوقتي. وبعد السطر ده، بدل ما تكتب `numpy.something` كل مرة، هتكتب `np.something` وخلاص.
 - `print(np.__version__)` — بيطلعك رقم نسخة المكتبة المثبتة عندك (زي `1.26.4` مثلاً). والسطر ده ليه فايدتين: الأولى إنه بيأكدلك إن التثبيت تمّ صح — لو اشتغل من غير خطأ يبقى كله تمام. والثانية إنك تعرف انت شغال على أنهي إصدار.
- ملحوظة صغيرة: الشرطتين اللي حوالين `__version__` دول **underscore مزدوج** من كل ناحية (يعني `_` مرتين قبل و `_` مرتين بعد)، مش شرطة واحدة.

KEY TAKEAWAYS

- NumPy ("Numerical Python") is an open-source, third-party Python module for working with arrays and matrices, plus the mathematical functions that operate on them.
- NumPy arrays beat Python lists on memory, speed, and ease of use, and support element-wise operations.
- Elements are stored **contiguously** — side by side in one block of memory — which is the underlying reason for those advantages.
- A Python list is **heterogeneous** (mixed types allowed); a NumPy array is **homogeneous** — all items must share one type.
- Because the type is fixed, the exact storage size an array needs is known in advance.
- Arrays are indexed from 0. Install with `pip install numpy` and import as `import numpy as np`.

الخلاصة في سطور

- NumPy = مكتبة خارجية Open Source في بايثون، تتعامل مع المصفوفات ومعها دوال رياضية جاهزة.
- ال **Array** أحسن من ال **List** في: الذاكرة، السرعة، سهولة الاستخدام، ودعم ال Element-Wise.
- السبب الحقيقي في ده كله إن العناصر متخزنة **ورا بعضها في الذاكرة** (Contiguous).
- ال **Heterogeneous** List (أنواع مختلفة)، وال **Homogeneous** Array (نوع واحد بس).
- وعشان النوع ثابت، NumPy عارفة **المساحة المطلوبة** من قبل ما تعمل المصفوفة.
- الترقيم بيبدأ من 0، والتثبيت بـ `pip install numpy`، والاستدعاء بـ `import numpy as np`.

وكده نكون خلّصنا الدرس الأول بحمد الله

راجع الخلاصة اللي فوق مرة كمان، خُذ بريك صغير — ونشوفك في اللي بعده.

END OF LESSON 01